Adaptive Two-Tier Route Prediction on Dynamic Graphs

A Transparent Multi-Model Framework and an Aircraft Next-Node Prototype

Krish Parikh
Independent Researcher, Ontario, Canada

15 August 2026

Preprint / working paper; not peer reviewed

Abstract

This paper documents an adaptive route-prediction framework conceived as a separation between candidate generation and candidate selection. Tier 1 represents movement on a dynamic directed graph, constructs feasible alternatives, and scores each candidate with interpretable signals including observed transitions, node frequency, destination progress, geometric resistance, weather, graph structure, and revisit penalties. Tier 2 uses a small neural network to rank Tier 1 candidates. Verified outcomes can subsequently update graph-edge weights and retrain the selector. The framework also specifies a broader contextual resistance layer for terrain, exposure, access, financial feasibility, allied or safe locations, and other mission constraints; those broader variables are design requirements and are not claimed as implemented in the aircraft prototype.

The prototype used public aircraft state observations and aviation-weather data stored in Google BigQuery. Its recorded training run contained 545 candidate rows, including 432 rows labelled as the observed next-node candidate. The fixed-weight multi-model score achieved 88.66% top-candidate accuracy when evaluated over the complete candidate table. A one-hidden-layer neural selector achieved 94.78% on its training split and 90.80% on 87 validation predictions. These scopes differ, so the numbers do not establish a clean improvement over the baseline. Live inference was not demonstrated: the live query and a nearest-historical-node fallback returned zero candidate rows on the available stale data. The result is therefore evidence of a working retrospective next-node prototype, not evidence of production route prediction, military readiness, or 98% accuracy.

Keywords: dynamic graphs, route prediction, next-node prediction, multi-model ensemble, neural selector, node frequency, adaptive graph, aircraft trajectories, terrain cost, uncertainty calibration

1 Scope, contribution, and provenance

The central contribution is the integration of several established mathematical tools into one auditable generate–select–adapt architecture. This paper does *not* claim to invent shortest-path algorithms, Markov chains, Monte Carlo reasoning, graph centrality, or neural networks. Shortest-path and loopless K-path methods are established results [1, 2]. The project contribution is the proposed division of labour between these methods, the explicit contextual assumptions, the expected-cost bound in section 4, the adaptive feedback structure, and an initial aircraft implementation.

The author's substantive project contributions are:

1. framing route prediction around a known current node, a known or uncertain goal, generalized least resistance, and soft no-backtracking/revisit assumptions;
2. requiring node and transition frequency, candidate enumeration, elimination of infeasible alternatives, multiple plausible paths, and confidence-aware Top-K output rather than a single unexplained route;

3. defining a two-tier system in which interpretable graph models generate candidate evidence and a learned selector ranks that evidence;
4. specifying contextual weights for weather, terrain difficulty, allies or safe locations, financial and access feasibility, exposure, constraints, and prior behaviour;
5. proposing behaviour-pattern learning from verified transitions and feedback into both graph weights and the selector;
6. proposing contour-map-to-graph terrain costs for military mobility and search-and-rescue applications, together with transfer to networking, logistics, infrastructure, and emergency movement; and
7. directing, configuring, executing, and evaluating the aircraft prototype and its cloud data pipeline.

OpenAI ChatGPT was used as an implementation and writing assistant. It translated project requirements into portions of the Python/SQL prototype, helped formalize equations and documentation, and drafted language that the author reviewed. In particular, the gradient-based terrain equations are a mathematical formalization of the author's contour-map requirement rather than a claim that the author invented vector calculus. The author is responsible for the framework choices, the reported project facts, and the final manuscript. This disclosure matters because the project should not be represented as entirely handwritten code or unaided prose.

1.1 Claim-status convention

To prevent plans from being confused with completed work, this paper uses three statuses:

Table 1: Meaning of implementation-status labels.

Status	**Meaning**
Implemented	Present in the archived prototype code or recorded cloud tables.
Measured	Implemented and accompanied by a preserved numerical run result.
Specified extension	Part of the framework design but not demonstrated in the aircraft run.

2 Problem formulation

At time t, let the movement environment be a directed graph

$$G_t = (V_t, E_t), \tag{1}$$

where a node $v \in V_t$ is a location or state and a directed edge $e = (u, v) \in E_t$ is a feasible transition. The graph is dynamic because edges may appear, disappear, or change cost as weather, access, congestion, threat, capacity, mission objectives, or data quality change.

The observed state is denoted x_t. It can include current node s_t, previous node s_{t-1}, position, speed, environmental conditions, a partial movement history, and a distribution over possible goals. When the goal is not known, the model must maintain

$$\mathbb{P}(g \mid x_t), \qquad g \in \mathcal{G}_t, \tag{2}$$

rather than silently treating a proxy destination as ground truth.

There are two related prediction tasks:

1. **next-node prediction**: rank $v \in \mathcal{N}^+(s_t)$ for the next observation horizon; and

2. **path prediction**: rank complete or partial feasible paths $\pi = (s_t, v_1, \ldots, g)$.

The recorded aircraft experiment is a next-node experiment. Full multi-hop path enumeration is part of the framework but was not validated by the recorded run.

3 Behavioural and operational assumptions

The framework begins from explicit assumptions instead of hiding them inside a learned score.

3.1 Known start, uncertain goal

The current observed position is the start state. A tracked object may know its own destination while the observer does not. The system must therefore represent the cases *known goal*, *inferred goal*, and *unknown goal*. The aircraft prototype did not contain actual flight-plan destinations; it used the nearest matched airport as a weak proxy. That proxy is an implementation expedient, not evidence that the destination was known.

3.2 Generalized least resistance

The least-resistance route need not be the physically shortest route. Define the contextual resistance of an edge as

$$\begin{aligned} R_t(e \mid x_t) = {} & \beta_{\text{time}}\, r_e^{\text{time}} + \beta_{\text{dist}}\, r_e^{\text{dist}} + \beta_{\text{weather}}\, r_e^{\text{weather}} + \beta_{\text{terrain}}\, r_e^{\text{terrain}} \\ & + \beta_{\text{exposure}}\, r_e^{\text{exposure}} + \beta_{\text{access}}\, r_e^{\text{access}} + \beta_{\text{finance}}\, r_e^{\text{finance}} + \beta_{\text{revisit}}\, r_e^{\text{revisit}} \\ & - \beta_{\text{safe}}\, a_e^{\text{safe}} - \beta_{\text{habit}}\, a_e^{\text{habit}}, \end{aligned} \tag{3}$$

where resistance terms increase cost, attraction terms reduce cost, and all inputs must be scaled before they are combined. Safe-location, relational, and financial data are only meaningful when lawfully obtained, relevant, current, and auditable. Equation (3) is a general framework specification. The aircraft prototype implemented only a subset: historical transition and node frequency, destination progress, geometry, weather, graph features, and an immediate-backtracking penalty.

3.3 Backtracking and revisits are soft constraints

An immediate return to the node just left is downweighted because it often provides less forward information. It is not absolutely forbidden: a blockage, weather change, dead end, deception, new goal, or safety requirement can justify reversal. The prototype used

$$S_{\text{elim}}(v) = \begin{cases} 0.25, & v = s_{t-1}, \\ 1, & \text{otherwise.} \end{cases} \tag{4}$$

Longer-term revisit penalties should decay. A generic exponential kernel requested for the framework is

$$k(z) = \exp(-\alpha z), \qquad \alpha > 0, \tag{5}$$

where z must be defined explicitly as elapsed time, path distance, or another normalized separation. The archived aircraft selector did not implement this longer-term decay kernel.

3.4 Constraints and process of elimination

Candidate construction should first remove transitions that are impossible under explicit hard constraints, then downweight transitions that are merely undesirable. Hard constraints can include closed edges, range limits, prohibited regions, capacity, or legal boundaries. Soft constraints can include risk, congestion, terrain, cost, or low historical support. This separation prevents a low probability from being mistaken for impossibility.

4 Expected route-cost bound

The statement that the expected route lies between the shortest and longest route is mathematically meaningful only after specifying a finite feasible set and a scalar cost. Let

$$\mathcal{P}_t(s, g; H) \tag{6}$$

be the nonempty set of feasible loopless paths from s to g satisfying the current constraints and a maximum horizon H. The horizon or loopless requirement is essential: if positive-cost cycles are allowed indefinitely, a longest path need not exist.

For a scalar path cost

$$C_t(\pi) = \sum_{e \in \pi} R_t(e \mid x_t), \tag{7}$$

define

$$C_{\min} = \min_{\pi \in \mathcal{P}_t} C_t(\pi), \qquad C_{\max} = \max_{\pi \in \mathcal{P}_t} C_t(\pi). \tag{8}$$

If the model assigns a valid distribution $p_t(\pi \mid x_t)$ over the feasible paths, then

$$\mathbb{E}[C_t(\Pi) \mid x_t] = \sum_{\pi \in \mathcal{P}_t} p_t(\pi \mid x_t) C_t(\pi), \tag{9}$$

$$C_{\min} \le \mathbb{E}[C_t(\Pi) \mid x_t] \le C_{\max}. \tag{10}$$

The inequalities are strict when the distribution assigns positive probability to non-extreme costs and is not concentrated entirely on a minimum- or maximum-cost path.

This is a valid bound, but it is not by itself a prediction algorithm: every bounded random variable lies between its extrema. It also applies to *cost or length*, not literally to the geometric location of an entire route. If each path is represented by an edge-incidence vector χ_π, then

$$\mathbb{E}[\chi_\Pi \mid x_t] = \sum_{\pi} p_t(\pi \mid x_t) \chi_\pi \in \operatorname{conv}\{\chi_\pi : \pi \in \mathcal{P}_t\}, \tag{11}$$

but the expected vector need not itself be a valid discrete path. A deployable system must still rank actual feasible paths.

5 Two-tier architecture

Tier 1 preserves structured alternatives and the component evidence behind them. Tier 2 does not invent routes; it ranks candidates supplied by Tier 1. This division permits human review, ablation of individual signals, and explicit hard constraints outside the neural model.

For full path prediction, Tier 1 should generate direct and multi-hop loopless paths using bounded enumeration or a K-shortest-loopless-path algorithm [2]. The current implementation instead expands observed outgoing historical transitions and therefore predicts the next discretized node.

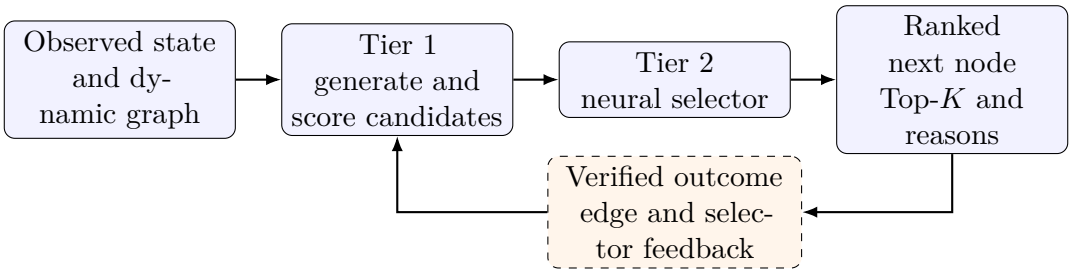


Figure 1: Generate–select–adapt architecture. The prototype's first measured run implemented Tier 1 candidate scores and a Tier 2 selector. A later script implemented the feedback structure, but no preserved numerical feedback-run result is claimed here.

5.1 Recorded candidate construction and target formation

The measured system is a grouped ranking model. For an observation identified by `aircraft_position_key`, let u be the current discretized node and let v^+ be the next node subsequently observed for that aircraft. The candidate set is

$$\mathcal{C}(u) = \{v : (u, v) \text{ occurs in the historical transition table}\}. \tag{12}$$

The candidate table contains one row for each $v \in \mathcal{C}(u)$ and attaches the binary target

$$y(u, v) = \mathbb{I}(v = v^+). \tag{13}$$

Thus the neural network does not directly output a coordinate or generate an edge. It estimates a score for each already generated candidate row; the prediction is the highest-scoring row within the position-key group.

This construction has an important closed-world limitation. If the current node u has no row in the historical transition table, then $\mathcal{C}(u)$ is empty and neither Tier 1 nor Tier 2 can make a prediction. If the true next edge has never been observed, the selector cannot propose it. The zero-candidate live result in section 12.1 is therefore a candidate-generation failure, not a neural-network classification error.

5.2 Complete recorded training and inference algorithm

The preserved implementation can be stated without collapsing the two tiers:

Offline table construction and training.

1. Order repeated aircraft observations and map each latitude–longitude pair to the 0.25° node in (14). Retain the current node, previous node, and observed next node where available.
2. Count directed transitions $u \to v$, node visits, and outgoing transition entropy. Build context-conditioned adaptive-transition and precomputed centrality tables.
3. For every eligible position key, join its current node to every historically observed outgoing transition, producing the candidate set in (12).
4. Attach kinematic, airport-proxy, and weather context; calculate the component scores in section 6; and calculate the fixed Tier 1 score in (27).
5. Attach the correctness label in (13). Candidate rows without a position key, candidate node, or observed next node are excluded from training.

6. Shuffle unique position keys with seed 42 and assign the first 80% to training and the remainder to validation. All candidate rows for one position key stay in the same split.
7. Replace nonnumeric, infinite, or missing feature values by zero. Estimate feature means and population standard deviations from training rows only; replace any zero standard deviation by one.
8. Initialize and train the 20–16–1 network described in section 7, then score all candidate rows using the training-set normalization parameters.
9. Within each position key, sort candidates by neural score, retain rank one, compare it with the observed next node, and write scores, verification records, summary metrics, normalization values, and network weights.

Attempted live inference.

1. Select the two most recent observations for each aircraft inside a 24-hour window; use them as current and previous nodes.
2. Use the nearest matched airport as the estimated destination because actual flight-plan destination data were unavailable.
3. Expand the current node through the historical transition table and join node-frequency, entropy, adaptive-transition, centrality, and weather data.
4. Recalculate all Tier 1 features, fill missing neural features with zero, normalize using the stored training means and standard deviations, and evaluate the stored network.
5. Rank candidates within each position key and write the top candidate as a 60-second-horizon prediction. In the preserved run, the expansion step returned no candidate rows, so the later steps produced no live prediction.

6 Tier 1: graph statistics and interpretable candidate scores

6.1 Node construction

The aircraft code discretized latitude and longitude into 0.25° cells:

$$v(\phi, \lambda) = \left(\frac{\operatorname{round}(4\phi)}{4}, \frac{\operatorname{round}(4\lambda)}{4} \right). \tag{14}$$

This coarse grid reduces sparsity but also limits spatial precision. One quarter degree is not a constant physical distance in longitude, and it is too coarse for terminal operations or tactical movement. The grid size is a prototype choice, not a universal design constant.

6.2 Transition and node frequency

Let n_{uv} be the observed number of transitions from node u to node v, and $n_u = \sum_v n_{uv}$. The empirical Markov score is

$$S_{\text{Markov}}(u, v) = \widehat{P}(v \mid u) = \frac{n_{uv}}{n_u}, \tag{15}$$

subject to whatever data-quality filtering created the transition table. A smoothed form should be used when counts are small. Node frequency supplies a separate global prior

$$S_{\text{freq}}(v) = \frac{n_v}{\sum_w n_w}. \tag{16}$$

Frequency is evidence of habit or network centrality; it is not proof of future intent. A production implementation should condition it on time, entity, phase, and context and should apply decay such as (5).

The outgoing transition entropy at u is

$$H(u) = -\sum_{v \in \mathcal{N}^+(u)} \widehat{P}(v \mid u) \log \widehat{P}(v \mid u). \tag{17}$$

The implemented stochastic-routing score penalizes uncertain source nodes:

$$S_{\text{stochastic}}(u, v) = \frac{\widehat{P}(v \mid u)}{1 + H(u)}. \tag{18}$$

These three statistics answer different questions. The Markov score asks, "given this source node, how often did this edge occur?" The node-frequency score asks, "how common is the proposed destination node across the recorded graph?" The stochastic score retains the edge probability but reduces it when the source has many similarly plausible exits. Markov and stochastic scores are local to u; node frequency is a global prior. A frequent hub can therefore receive a high frequency score even when a particular incoming edge is weak.

No pseudocount, Bayesian prior, or minimum-support threshold is visible in the preserved predictor. An edge observed once can consequently receive a high empirical probability when its source node has little data. A production version should store n_{uv} and n_u with every score, apply smoothing, and expose a low-support warning instead of presenting every empirical probability with equal confidence.

6.3 Geometry, destination progress, and motion consistency

Let $d(v, g)$ be great-circle distance in kilometres. The code used

$$S_{\text{shortest}}(v) = \frac{1}{1 + d(v, g)}. \tag{19}$$

Because g was the nearest-airport proxy, this is not a verified flight-plan shortest-path score.

With $d_c = d(u, g)$ and $d_n = d(v, g)$, destination progress was scored as

$$S_{\text{progress}}(v) = \begin{cases} 0, & d_c = 0 \text{ or unavailable}, \\ \min\left(1, 0.5 + \dfrac{d_c - d_n}{d_c}\right), & d_n < d_c, \\ \dfrac{0.25}{1 + d_n / d_c}, & d_n \geq d_c. \end{cases} \tag{20}$$

For a 60-second prediction horizon, the expected travel distance was computed from ground speed, and motion consistency was

$$S_{\text{dynamic}}(v) = \frac{1}{1 + |d(u, v) - d_{\text{expected}}|}. \tag{21}$$

The implementation used BigQuery geographic distance for $d(\cdot, \cdot)$ and

$$d_{\text{expected}} = \text{speed}_{\text{kt}} \frac{1.852 \times 60}{3600} = \text{speed}_{\text{kt}} \frac{1.852}{60} \text{ km}. \tag{22}$$

The so-called shortest-path score is therefore only inverse distance from the candidate node to the destination proxy. It does not run Dijkstra's algorithm, sum edge costs, or prove that the candidate lies on a globally shortest route. The destination-progress score compares the candidate with the current node, whereas the dynamic score compares the candidate step length with one minute of travel at the recorded ground speed. Heading, vertical rate, turn limits, and aircraft performance envelopes were not included in the preserved 20-feature selector. When speed is missing, the live query substitutes zero, which makes shorter candidate steps look more motion-consistent.

6.4 Context-conditioned adaptive transition model

The adaptive table supplies a transition probability conditioned on the source node and three context variables. The live predictor constructed altitude bands as

$$b(h) = \begin{cases} \text{low}, & h < 5{,}000 \text{ ft}, \\ \text{medium}, & 5{,}000 \le h < 18{,}000 \text{ ft}, \\ \text{high}, & 18{,}000 \le h < 35{,}000 \text{ ft}, \\ \text{very high}, & h \ge 35{,}000 \text{ ft}, \\ \text{unknown}, & h \text{ missing}, \end{cases} \tag{23}$$

and airport phase as terminal within 25 km of the nearest airport, terminal area from 25 to below 100 km, en route at 100 km or more, and unknown when airport distance was missing. The fourth context field was the VFR/MVFR/IFR/LIFR category or unknown.

Semantically, the model is

$$S_{\text{adaptive}}(u, v) = \widehat{P}(v \mid u, b(h), p_{\text{airport}}, c_{\text{flight}}) \,. \tag{24}$$

Inference joins on all conditioning fields and v. When no context-specific row matches, the code falls back to the unconditional transition probability $\widehat{P}(v \mid u)$. The preserved predictor and trainer establish this join and fallback behaviour; the archive inspected for this paper did not preserve the exact table-building SQL, smoothing rule, or minimum support for (24). Those details are therefore not reconstructed here.

6.5 Graph-centrality model

The graph feature is candidate-node specific. Inference joins `route_graph_centrality_v2` on v and uses the resulting `graph_centrality_score` directly as $S_{\text{centrality}}(v)$. Its purpose is to represent structural importance that is not identical to raw node frequency: a strategically positioned connector can matter even if it is not the most visited node.

The exact centrality construction was not present in the preserved predictor, selector, or feedback scripts inspected for this manuscript. It would therefore be false to label this value specifically as degree, betweenness, closeness, eigenvector centrality, or PageRank. The paper treats it only as a precomputed graph-centrality feature. Reproducible release requires the missing construction query, its normalization, its direction convention, and the graph snapshot on which it was calculated.

6.6 Node elimination and soft backtracking

The node-elimination model is the operational form of (4). A non-backtracking candidate receives 1 and an immediate return to the previous node receives 0.25. Because this score enters (27) with weight 0.05, its maximum contribution is 0.05 and its immediate-backtrack contribution is 0.0125. The difference, 0.0375, is deliberately a penalty rather than a hard removal. Longer loops, repeated corridors, and time-decayed revisits were not detected by this rule.

6.7 Weather-resistance score

The prototype mapped flight category and wind speed to hand-set factors:

$$S_{\text{weather}} = f_{\text{category}} \, f_{\text{wind}}, \tag{25}$$

where f_{category} was $1.00, 0.75, 0.45, 0.20$ for VFR, MVFR, IFR, and LIFR respectively (or 0.50 when unknown). The wind factors were 1.00 below 15 kt, 0.75 from 15 to below 30 kt, 0.50 from 30 to below

Table 2: Implemented fixed Tier 1 weights. They sum to 1.00.

Signal	Weight	Signal	Weight
Markov transition	0.18	Node frequency	0.12
Stochastic routing	0.10	Shortest/destination distance	0.12
Destination progress	0.12	Dynamic motion consistency	0.10
Graph centrality	0.08	Adaptive graph probability	0.10
Node elimination	0.05	Weather resistance	0.03

45 kt, and 0.25 at or above 45 kt (or 0.75 when missing). These are heuristic prototype values and were not aerodynamically validated.

Multiplication makes the two penalties interact. For example, an IFR observation with 30–44 kt wind receives $0.45 \times 0.50 = 0.225$. The score is attached to the observation-level weather context and is the same for every outgoing candidate in that group unless candidate-specific weather is introduced. It can therefore change the absolute Tier 1 score without necessarily changing the order of candidates from the same observation. Directional wind, route-specific forecasts, convective weather, icing, turbulence, ceilings, airspace restrictions, and aircraft-type limits were not modelled by this heuristic.

6.8 Monte Carlo proxy and combined score

The field named `monte_carlo_chain_proxy_score` was not the result of repeated Monte Carlo simulation. In the prototype query it was the deterministic mixture

$$S_{\text{MCproxy}} = 0.50 S_{\text{Markov}} + 0.30 S_{\text{shortest}} + 0.20 S_{\text{stochastic}}. \tag{26}$$

The name should therefore be interpreted as a proxy feature, not as evidence that a trajectory simulator was run.

The Tier 1 combined score used the fixed weights in table 2.

Thus

$$\begin{aligned} S_{\text{Tier1}} = {} & 0.18 S_{\text{Markov}} + 0.12 S_{\text{freq}} + 0.10 S_{\text{stochastic}} + 0.12 S_{\text{shortest}} \\ & + 0.12 S_{\text{progress}} + 0.10 S_{\text{dynamic}} + 0.08 S_{\text{centrality}} + 0.10 S_{\text{adaptive}} \\ & + 0.05 S_{\text{elim}} + 0.03 S_{\text{weather}}. \end{aligned} \tag{27}$$

These weights are an interpretable starting prior. The recorded experiment did not establish that they are optimal.

6.9 Score scale, interactions, and ranking

Most implemented component scores were designed to lie in $[0, 1]$, but the fixed combination assumes that the precomputed centrality and adaptive fields are on comparable scales. The Tier 1 SQL does not standardize the ten components before applying (27), does not learn the weights, and does not clip the final sum. Consequently, a component's practical influence depends on both its stated weight and its empirical range. The Tier 2 network later standardizes its 20 inputs, which changes the scale issue but not the information contained in a feature.

Tier 1 produces a ranking, not a normalized probability distribution. Tier 2 likewise produces independent sigmoid scores for candidate rows; those scores do not sum to one within an observation. No domain-defined secondary tie-breaker was preserved in the Python group-ranking step. A reproducible operational version should define a stable order, for example neural score, then support count, then candidate identifier, and should separately calibrate group probabilities.

6.10 Probabilistic alternatives discussed but not measured

Two additional probability constructions were considered during development but were not part of the preserved measured run. First, a genuine Monte Carlo route model would repeatedly sample

$$V_{j+1} \sim P_t(\cdot \mid V_j, h_j, x_t) \tag{28}$$

until a goal, horizon, or failure state is reached, then estimate a path probability or utility from the empirical rollout distribution. This is different from the deterministic proxy described above.

Second, candidate utilities could be converted to a multinomial-logit distribution,

$$P(\pi_i \mid x_t) = \frac{\exp(U_i(x_t))}{\sum_{j=1}^{K} \exp(U_j(x_t))}, \tag{29}$$

with a nested-logit extension when routes share major corridors and should not be treated as independent alternatives. These models provide interpretable probabilistic baselines against which the neural selector should be compared. Neither logit model was implemented in the recorded aircraft selector.

6.11 Specified full-path algorithm

The broader algorithm requested for the project operates on complete alternatives, but it must be identified as a specified extension rather than part of the measured aircraft result. At time t it would execute the following procedure:

1. Construct G_t from the latest allowed nodes, edges, closures, costs, and contextual constraints. Remove only transitions that violate hard constraints.
2. Form a goal set $\mathcal{G}_t$. Use a point mass for a known goal or maintain $P(g \mid x_t)$ when the observer is uncertain.
3. For each goal g, use Dijkstra's algorithm as the least-cost subroutine and Yen's method or another bounded method to generate at most K loopless feasible paths with horizon H [1, 2].
4. For each path π, aggregate edge resistance $C_t(\pi)$, log transition support $\sum_{(u,v)\in\pi} \log \widehat{P}(v \mid u)$, node-frequency and recency terms, destination progress, terrain cost, weather cost, revisit penalties, and any legally authorized domain constraints.
5. Preserve every component separately. Compute either an interpretable weighted path utility, a learned path-selector score, or both. A learned selector must be trained on complete-path outcomes; the existing next-node network cannot be relabelled as a full-path model.
6. If probabilities are required, normalize utilities across the generated feasible set using a tested logit or nested-logit model, or estimate them from genuine Monte Carlo rollouts. Report Top-K, entropy, and calibration rather than only the maximum.
7. Output actual feasible paths ordered by posterior score. Report $\widehat{\pi}_{\mathrm{MAP}}$, alternatives, component explanations, and the expected cost. Verify that the expected value satisfies (10).
8. When the true route becomes known, update transition counts and context-conditioned tables using a time-stamped rule, record prediction error, detect drift, and retrain only under a versioned evaluation protocol.

This full-path procedure resolves a common ambiguity in the project language. "Between the shortest and longest path" refers to the expected scalar cost in (10); the algorithm must still return one or more discrete feasible paths. It cannot return the arithmetic average of route coordinates and call that average a route.

7 Tier 2: neural candidate selector

For each candidate, the selector receives the 20 features shown in table 3. Numeric missing values are replaced by zero after conversion. Means and standard deviations are learned from the training split, with zero standard deviations replaced by one.

The recorded network is a binary candidate-correctness model with one hidden layer:

$$h_i = \mathrm{ReLU}(\widetilde{x}_i W_1 + b_1), \tag{30}$$

$$q_i = \sigma(h_i W_2 + b_2), \tag{31}$$

where $\widetilde{x}_i \in \mathbb{R}^{20}$ is the normalized feature vector, $W_1 \in \mathbb{R}^{20\times 16}$, and $W_2 \in \mathbb{R}^{16\times 1}$. Candidates are ranked by q_i within each `aircraft_position_key`; the top candidate becomes the predicted next node.

The architecture contains 353 trainable parameters: 20×16 input weights, 16 hidden biases, 16 output weights, and one output bias. Both weight matrices were initialized independently from a zero-mean normal distribution with standard deviation 0.1, and biases were initialized to zero. The sigmoid input was clipped to $[-40, 40]$ for numerical stability. ReLU supplies a piecewise-linear interaction layer, allowing, for example, a transition-frequency signal to be interpreted differently at different speeds or weather states.

Training used 16 hidden units, 1,500 epochs, learning rate 0.01, and random seed 42. The positive-class weight was

$$w_+ = \max\left(1, \frac{N_-}{N_+}\right), \tag{32}$$

and the implementation optimized a corresponding weighted binary cross-entropy gradient. There was no recorded dropout, regularization, early stopping, or hyperparameter search.

More explicitly, with row weight $\omega_i = w_+$ for a correct candidate and $\omega_i = 1$ otherwise, the logged objective was

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} \omega_i \left[y_i \log(q_i + \epsilon) + (1 - y_i)\log(1 - q_i + \epsilon)\right], \tag{33}$$

with $\epsilon = 10^{-9}$. Training used full-batch plain gradient descent: every epoch evaluated all training rows, back-propagated $(q_i - y_i)\omega_i$, averaged the gradients by the number of rows, and subtracted learning rate 0.01. Loss was printed every 250 epochs. No Adam optimizer, mini-batching, shuffling between epochs, or validation-based stopping appears in the preserved script.

Although q_i is produced by a sigmoid, it is not a calibrated mutually exclusive next-node probability. Each candidate is scored independently under the binary objective, so generally

$$\sum_{v\in\mathcal{C}(u)} q(u, v) \neq 1. \tag{34}$$

The preserved system uses q_i only as a ranking score. A probability claim would require group normalization and calibration on held-out data.

Unique position keys were shuffled and split 80/20 into training and validation sets. This prevents candidate rows for the same position key from being split across sets, but it does not separate entire aircraft or future time blocks. The validation accuracy can therefore be optimistic if the same aircraft, corridor, or time regime occurs in both partitions.

There is a second possible source of optimism: the preserved evidence does not establish that the historical transition, frequency, adaptive, and centrality tables were rebuilt using training observations only. If those graph statistics included observations from validation time, labels were not directly passed to the network, but future structural information could still leak into validation features. A publishable test must construct every derived table inside each training fold or use a strictly earlier time window.

8 Adaptive graph and two-network feedback extension

A later implementation script encoded a two-level feedback iteration. This is distinct from the overall two-tier architecture: the architecture is Tier 1 generation plus Tier 2 selection, whereas the feedback implementation happens to train two neural networks.

First, an edge network estimates candidate correctness q_{uv}^{edge} from the same 20 base features and converts it to a bounded multiplier

$$m_{uv} = 0.25 + 1.75 q_{uv}^{\text{edge}}, \tag{35}$$

so $m_{uv} \in (0.25, 2.00)$. The edge model is trained on candidate rows, after which its scores and multipliers are averaged by directed edge to create an edge-update table containing observation count, correct count, empirical edge accuracy, original scores, and geographic error.

At candidate level, Markov, stochastic, and adaptive-graph scores are multiplied by m_{uv}. The feedback proxy is then rebuilt as

$$S_{\text{MCproxy}}^{\text{fb}} = 0.50 S_{\text{Markov}}^{\text{fb}} + 0.30 S_{\text{progress}} + 0.20 S_{\text{stochastic}}^{\text{fb}}, \tag{36}$$

which differs slightly from the first live proxy because its middle term is destination progress rather than inverse destination distance. The fixed Tier 1 weights remain unchanged, but the three multiplied components can now exceed one, so the feedback combined score is not guaranteed to remain in $[0, 1]$.

Second, a selector network is retrained on 22 feedback features. They comprise the feedback-adjusted Markov, proxy, stochastic, adaptive, and combined scores; the unchanged frequency, shortest, progress, dynamic, centrality, elimination, and weather scores; the edge-network score and multiplier; and the same eight count, kinematic, weather, category, and airport-distance fields used in the original selector. Both feedback networks use one 16-unit ReLU layer, 1,200 epochs, and learning rate 0.01; seeds 11 and 22 are used for the edge and selector networks respectively.

The script writes updated edge, candidate, verification, performance, and model-weight tables. No preserved numerical output from a completed feedback run was available for this manuscript. Consequently, the feedback code is described as implemented, but no accuracy improvement from it is claimed.

The feedback script also exposes an evaluation defect that must be fixed before its results can be used. It normalizes and trains the edge updater on the complete candidate dataframe before creating the later 80/20 selector split. Edge-network outputs supplied to validation rows have therefore already been fitted using those rows' correctness labels. Even if a numerical validation result were recovered, it would be contaminated by this label feedback. The correct procedure is to train the edge updater on training keys only, generate out-of-fold feedback features for model selection, and reserve an untouched time- and aircraft-separated test set.

9 Pattern learning

The broader framework requires verified behaviour to update future priors without treating repetition as certainty. For a candidate node v, an interpretable blend is

$$\widehat{P}_{\text{blend}}(v) = c\, \widehat{P}_{\text{entity}}(v \mid h_t, x_t) + (1 - c)\, \widehat{P}_{\text{base}}(v \mid x_t), \tag{37}$$

where h_t is recent history and $c \in [0, 1]$ increases only when entity-specific evidence is sufficiently recent, numerous, and calibrated. Useful pattern signals include preferred corridors, reversals, loops, goal switching, response to closures or weather, and time between revisits.

Equation (37) is a design specification. The recorded first selector included transition frequency and an adaptive transition table, while the later feedback script updated edge weights. It did not demonstrate a separate sequence model, calibrated entity-specific confidence c, or drift detector.

10 Contour graphs and terrain difficulty

The terrain extension begins with contour lines or a digital elevation model and produces a continuous elevation surface $z(x, y)$. The local slope magnitude and directional grade are

$$s(x, y) = \|\nabla z(x, y)\|, \qquad g_{\hat{u}}(x, y) = \nabla z(x, y) \cdot \hat{u}, \tag{38}$$

where $\hat{u}$ is the direction of travel. Curvature or roughness can be derived from the Hessian H_z or from local elevation residuals. A node-level terrain index can then be defined as

$$T(v) = \eta_s s(v) + \eta_r r(v) + \eta_m m(v) + \eta_w w(v), \tag{39}$$

where r is roughness, m represents surface or mobility class, and w represents weather-induced terrain effects. The terrain contribution for an edge of geometric length L_e is

$$T(e) = \frac{1}{L_e} \int_e T(x, y)\, ds. \tag{40}$$

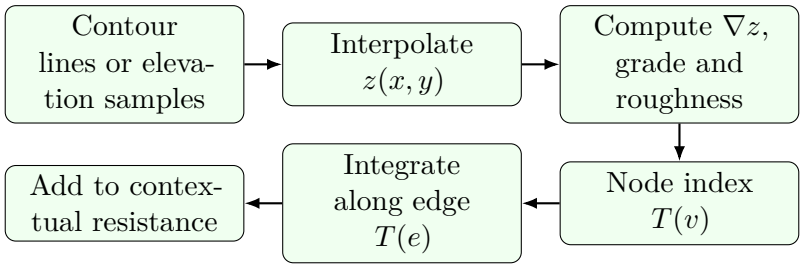


Figure 2: Specified contour-to-graph transformation. It was not implemented in the recorded aircraft prototype.

Coefficients must be calibrated separately for people, wheeled vehicles, tracked vehicles, and aircraft; terrain resolution and interpolation error must be carried into confidence. For search and rescue, terrain accessibility can be combined with last-known position, goal hypotheses, weather, and observed movement patterns. For military mobility, terrain is one decision-support input alongside access, obstacles, exposure, logistics, and human authorization. No field validation of (39)–(40) is claimed.

11 Data and implementation

11.1 Data sources and storage

The project used Google Cloud project `route-prediction-algorithm` and BigQuery dataset `route_prediction`. Aircraft state vectors came from the OpenSky Network's public data services; OpenSky describes its datasets as crowdsourced air-traffic surveillance data [3, 4]. Aviation weather came from the Aviation Weather Center data service, which documents machine-to-machine access to METAR and related products [5]. BigQuery served as the table and query layer [6].

Recorded development logs indicate two successful aircraft-state insertions of 100 rows each. Attempts involving certain flight or aircraft endpoints returned HTTP 400/403 responses. These logs do not establish a comprehensive or continuously sampled flight dataset. Weather matching and derived-table creation later produced the working retrospective candidate set.

The early schema also contained three groups of tables:

- movement data: `aircraft_positions` and `flight_segments`;

- reference and context: `airports`, `navaids`, `runways`, `weather`, and `airspace`; and
- outputs: `model_performance` and `route_predictions`.

At one checkpoint no `flight_segments` rows were available. The measured v2 graph therefore depended on transitions derived from repeated position observations rather than a validated table of complete flight segments. The airport, navaid, runway, and airspace tables were part of the intended data model; their contribution was not isolated in the recorded selector experiment.

11.2 Pipeline

The orchestration script records the following stages:

1. `ingest_opensky_states.py`;
2. `ingest_aviation_weather.py`;
3. `create_matched_aircraft_weather.py`;
4. `create_route_model_features.py`;
5. `create_route_backend_multimodel.py`;
6. `train_neural_model_selector.py`; and
7. either live inference or a no-time-filter stale-data/backtest path.

The archived predictor requires the tables in table 4. Several additional tables store scores, verification, performance, and serialized weights.

At one recorded checkpoint, `matched_aircraft_weather` had 569 rows with latest aircraft timestamp 8 July 2026 22:28:11, and `route_node_transitions_v2` had 375 rows. These are development snapshots, not stable dataset release statistics.

12 Recorded experiment

The preserved model version was `nn_selector_v2_20260712_200151`. The candidate table supplied 545 candidate rows, of which 432 were labelled as matching the observed next node. Evaluation ranked candidates separately for each position key and measured whether the top-ranked node matched the recorded next node.

The implementation also created kilometre-error columns using great-circle distance, but no preserved average, median, or 90th-percentile error values were available for this manuscript. No full-path accuracy, Top-K recall, Brier score, negative log likelihood, calibration curve, confidence interval, or external test set was preserved. A 98% accuracy claim would therefore be unsupported.

12.1 Live-inference failure

The live predictor filtered for observations within the prior 24 hours and attempted to join the current discretized node to historical outgoing transitions. It loaded zero live candidate rows. A fallback script disabled the time filter and permitted the nearest historical source node within 500 km, prioritizing exact matches and then proximity and transition volume; it still reported no candidates in the preserved run. This outcome is a project result and must be reported, not omitted.

The likely data-path problems visible in the implementation are stale observations, incomplete overlap between current nodes and the historical transition graph, and the absence of actual destination data. Until new observations produce candidates and predictions are verified prospectively, the system is a retrospective prototype.

Table 3: Features used by the recorded neural selector.

	Feature		Feature
1	Markov-chain score	11	Least-resistance weather score
2	Monte Carlo proxy score	12	Combined multi-model score
3	Node-frequency score	13	Transition count
4	Stochastic-routing score	14	Altitude (ft)
5	Shortest-path proxy score	15	Ground speed (kt)
6	Destination-progress score	16	Temperature (°C)
7	Dynamic-system score	17	Wind speed (kt)
8	Graph-theory/centrality score	18	Visibility (statute mi)
9	Adaptive-graph score	19	Encoded flight category
10	Node-elimination score	20	Nearest-airport distance (km)

Table 4: Core BigQuery tables referenced by the prototype.

Table	Role
`matched_aircraft_weather`	Aircraft observations joined to nearest-airport weather and position-derived features.
`route_node_transitions_v2`	Historical directed transitions, counts, and empirical probabilities.
`route_node_frequency_v2`	Global node-frequency prior.
`route_node_transition_entropy_v2`	Outgoing transition uncertainty and candidate count.
`route_adaptive_transitions_v2`	Transition probabilities conditioned on altitude band, airport phase, and flight category.
`route_graph_centrality_v2`	Precomputed graph-centrality feature.
`route_prediction_candidates_v2`	Candidate rows with component scores and observed next-node label.
`route_nn_model_weights_v2`	Serialized normalization parameters, weights, and recorded metrics.
`route_nn_candidate_scores_v2`	Candidate-level Tier 2 scores.
`route_nn_prediction_verification_v2`	Top-ranked prediction compared with the observed next node.
`route_live_predictions_v2`	Intended 60-second-horizon inference output.

Table 5: Recorded next-node results. Scopes differ and must not be treated as one controlled comparison.

Model/score	Scope	Predictions	Top-1 accuracy
Fixed Tier 1 combined score	Entire table	Not preserved here	88.66%
Neural selector	Training split	Not preserved here	94.78%
Neural selector	Validation split	87	90.80%

13 Evaluation requirements

Next-node accuracy alone is insufficient. For prediction horizon N,

$$\mathrm{Acc}_{\mathrm{node}} = \frac{1}{N}\sum_{t=1}^{N} \mathbb{I}(\widehat{v}_{t+1} = v_{t+1}). \tag{41}$$

Full paths should also be evaluated with exact-path match, directed-edge precision/recall or Jaccard similarity, normalized edit distance, geographic error, and destination accuracy. The known starting node must be excluded when reporting predicted-position accuracy.

For a discrete grid, the previously considered Manhattan node error is

$$d_1(\widehat{v}, v) = |\widehat{i} - i| + |\widehat{j} - j|, \tag{42}$$

while geographic applications should also report haversine or projected-metre error. For complete alternatives, the maximum-a-posteriori output is

$$\widehat{\pi}_{\mathrm{MAP}} = \arg\max_{\pi \in \mathcal{P}_t} P(\pi \mid x_t), \tag{43}$$

but MAP must be accompanied by Top-K recall and distribution entropy so a narrow win is not presented as certainty.

Probability-like outputs require calibration. Brier score [7], negative log likelihood, reliability diagrams, expected calibration error, and selective abstention should accompany Top-K recall. Neural scores are not automatically calibrated probabilities [8]. The recorded q_i values were used for ranking and were not calibrated.

A credible validation design should:

1. split by aircraft and time, not only by position key;
2. compare against simple baselines on the same held-out observations;
3. report sample counts and uncertainty intervals;
4. test missing weather, stale observations, unseen nodes, graph drift, and destination error;
5. evaluate multiple grid resolutions and horizons;
6. perform ablations of each Tier 1 signal and the neural selector; and
7. progress from historical replay to shadow-mode live testing before any operational influence.

14 Exploratory simulator and operator interface

Earlier project requirements included a simulator with random or user-selected start and goal nodes, a dynamic adjacency or transition matrix, an adjustable exponential-decay parameter α, and side-by-side Team A/Team B or model-comparison views. A technically useful interface would display the current

graph, feasible candidates, component scores, MAP route, Top-K distribution, entropy, confidence, predicted-versus-actual path, and error metrics while allowing the environment to change between steps.

This interface is a specified research and demonstration tool, not part of the recorded BigQuery accuracy result. Its purpose is to expose assumptions and compare models under identical states, not to fabricate operational realism. Any future implementation should save the random seed, graph version, parameters, and complete event log so a displayed scenario is reproducible.

15 Applications and transfer limits

15.1 Aircraft

For civil or special-mission aviation, the system could rank likely next airspace nodes or corridors using filed-route information, weather, aircraft constraints, traffic, and airspace status. The present prototype does not ingest a filed route and must not be presented as a flight-planning or collision-avoidance system.

15.2 Military and contested movement

The framework can represent air, land, or maritime movement when objectives include exposure, route denial, access, logistics, weather, deception, and mission priority in addition to travel time. It is a decision-support architecture, not autonomous authority. Operational constraints, safety rules, legal authorization, confidence thresholds, audit logs, and human review must remain outside and above the selector. The reported aircraft accuracy does not transfer to military movement.

15.3 Search and rescue

Search and rescue can combine last-known position, possible goals, terrain accessibility from section 10, weather, movement mode, and historical behaviour to rank likely next locations and accessible search corridors. Validation requires real field tracks, missing-person behavioural expertise, and explicit uncertainty; a contour-derived cost surface alone does not establish where a person went.

15.4 Networking and information technology

Routers, services, hosts, or dependencies can replace physical nodes. Digital resistance can combine latency, packet loss, congestion, capacity, failure risk, policy cost, and security exposure. Tier 1 can generate packet, dependency, or failover paths, while Tier 2 ranks the candidate that historically performs best under the current fault pattern. Retraining and domain-specific labels are mandatory.

15.5 Logistics, infrastructure, and emergency movement

The same separation can support distribution routes under closures, evacuation corridors as hazards spread, or restoration sequences on dependency graphs. Only the architecture transfers automatically; graphs, features, constraints, labels, loss functions, and acceptable risk do not.

16 Limitations

The current evidence has material limitations:

- The measured task was next-node classification, not complete route prediction.
- The candidate table was small and was derived from a limited, non-continuous public-data collection.

- Validation was a single random 80/20 split by position key, not an external, aircraft-held-out, or forward-time test.
- The baseline and validation accuracies were computed on different scopes.
- The destination was a nearest-airport proxy, not a verified intended destination.
- The 0.25° grid is coarse and geographically nonuniform.
- The weather factors and fixed ensemble weights were heuristic.
- The so-called Monte Carlo feature was a deterministic proxy, not a simulation result.
- Live and fallback inference produced zero candidate rows.
- Confidence calibration and full-path metrics were not measured.
- Contour terrain, finance, allies/safe locations, broader contextual resistance, full path enumeration, and domain transfers remain specified extensions.
- The prototype code and manuscript were produced with substantial AI assistance, requiring independent technical review before research or operational reliance.

17 Implementation roadmap

A defensible completion sequence is:

1. freeze and version a reproducible unclassified dataset with provenance and licensing notes;
2. ingest actual or reliably inferred destinations and preserve complete time-ordered trajectories;
3. repair live graph coverage and demonstrate nonzero prospective predictions;
4. replace candidate-only testing with aircraft-held-out and forward-time evaluation;
5. implement bounded loopless multi-hop path generation and the bound in section 4;
6. calibrate confidence and add Top-K, geographic, directed-edge, and sequence metrics;
7. conduct feature ablations and replace heuristic factors where learned or physically validated alternatives are justified;
8. implement time-decayed node frequency, pattern confidence, and drift detection;
9. implement and validate the contour-to-terrain pipeline separately on field movement data; and
10. use staged replay, shadow, and human-assist evaluation before any live decision-support claim.

18 Conclusion

The project establishes a clear technical direction and a working retrospective next-node prototype. Its strongest idea is architectural: generate multiple feasible, interpretable candidates; learn which evidence to trust; update both the graph and selector from verified outcomes; and retain explicit constraints and human control. The expected-cost inequality correctly bounds a probability-weighted route cost between the minimum and maximum costs of a finite feasible path set, but it does not replace route inference. The recorded 90.80% validation accuracy is encouraging on a small split, while the zero-candidate live result and missing destination truth define the current boundary of the evidence. The next research milestone is therefore not a larger claim; it is a reproducible prospective test that successfully generates, ranks, and verifies live candidates.

Data, code, and ethics statement

The prototype used public-source aircraft and aviation-weather data. This manuscript contains no classified information and does not claim access to military data or systems. Any use involving personal, financial, relational, surveillance, defence, or protected information requires lawful authority, purpose limitation, data-quality controls, auditability, and human accountability. The development archive should be reviewed for third-party terms and data licences before public release. The author reports that the project was developed independently in a personal capacity without CAF/DND resources, systems, equipment, funding, or non-public data; that statement has not been independently audited in this manuscript.

References

[1] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1: 269–271, 1959. doi: 10.1007/BF01386390.

[2] Jin Y. Yen. Finding the k shortest loopless paths in a network. *Management Science*, 17(11):712–716, 1971. doi: 10.1287/mnsc.17.11.712.

[3] OpenSky Network. Opensky network data. https://opensky-network.org/data, 2026. Accessed 15 August 2026.

[4] Martin Strohmeier, Xavier Olive, Jannis L"ubbe, Matthias Sch"afer, and Vincent Lenders. Crowdsourced air traffic data from the opensky network 2019–2020. *Earth System Science Data*, 13:357–366, 2021. doi: 10.5194/essd-13-357-2021.

[5] Aviation Weather Center. Data api. https://aviationweather.gov/data/api/, 2026. Accessed 15 August 2026.

[6] Google Cloud. Bigquery documentation. https://cloud.google.com/bigquery/docs, 2026. Accessed 15 August 2026.

[7] Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950. doi: 10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.

[8] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330, 2017. URL https://proceedings.mlr.press/v70/guo17a.html.

A Model inventory

Table 6: Complete model inventory and claim boundary.

Model	Status	Input and output	Role and principal limitation
Hard-constraint filter	Specified	Allowed graph and operational rules → feasible edges	Removes impossibilities before scoring; no complete operational rule engine was implemented.
Markov transition	Implemented	$n_{uv}, n_u \rightarrow \widehat{P}(v \mid u)$	Local historical edge tendency; sparse counts and unseen edges are major failures.
Node frequency	Implemented	Global node visits → $S_{\text{freq}}(v)$	Habit or hub prior; can confuse popularity with current intent.
Entropy-adjusted stochastic	Implemented	Markov probability and $H(u) \rightarrow S_{\text{stochastic}}$	Downweights ambiguous source nodes; inherits Markov data bias.
Shortest/distance proxy	Implemented	Candidate-to-proxy-goal distance → $1/(1+d)$	Rewards proximity; not an actual shortest-path calculation.
Destination progress	Implemented	Current and candidate goal distances → S_{progress}	Rewards relative movement toward proxy goal; unreliable when the goal proxy is wrong.
Dynamic motion	Implemented	Ground speed and candidate step length → S_{dynamic}	Tests one-minute distance consistency; omits heading and performance constraints.
Adaptive transition	Implemented input	Source, candidate, altitude band, airport phase, flight category → conditional transition probability	Context-specific edge prior with unconditional fallback; construction SQL and support rule were not preserved.
Graph centrality	Implemented input	Candidate node → precomputed centrality score	Represents structural importance; the exact centrality definition was not preserved.
Node elimination	Implemented	Previous node and candidate → 0.25 or 1	Soft immediate-backtrack penalty; does not detect longer revisits.
Weather resistance	Implemented	Flight category and wind band → product factor	Simple observation-level heuristic; not route- or aircraft-specific.
Monte Carlo proxy	Implemented	Markov, distance/progress, stochastic scores → fixed mixture	Deterministic feature despite its name; no trajectories are sampled.
Fixed Tier 1 ensemble	Measured	Ten component scores → S_{Tier1}	Auditable baseline; weights were hand set and not tuned in a controlled comparison.
Neural selector	Measured	20 normalized candidate features → q_i	Learns nonlinear ranking interactions; output is not a calibrated group probability.
Edge feedback network	Implemented code	20 features and candidate label → q^{edge}, m_{uv}	Reweights selected graph signals; preserved code trains on all rows and contaminates later validation.
Feedback selector	Implemented code	22 adjusted features → candidate score	Retrains after edge adjustment; no defensible preserved performance result.
Entity-pattern blend	Specified	Recent entity history and base probability → $\widehat{P}_{\text{blend}}$	Allows habits and drift; no calibrated sequence learner was demonstrated.

Model	Status	Input and output	Role and principal limitation
Genuine Monte Carlo route model	Specified	Transition model and stopping rules → rollout distribution	Represents multi-step uncertainty; not implemented in the recorded run.
Multinomial/nested logit	Specified	Candidate utilities → normalized alternative probabilities	Interpretable probabilistic baseline; not implemented or calibrated.
K-path generator	Specified	Dynamic constrained graph and goal set → loopless paths	Required for full-route prediction; current prototype generates next nodes only.
Contour/terrain model	Specified	Elevation, slope, roughness, mobility and weather → $T(v), T(e)$	Extends resistance to field movement; no terrain dataset or field validation was used.

B Evidence and status matrix

Table 7: What the current archive supports.

Component	Status	Evidence or limitation
Dynamic graph and 0.25° nodes	Implemented	Predictor query constructs discretized nodes and historical transitions.
Node and transition frequency	Implemented	Dedicated frequency and transition tables are consumed by inference.
Fixed multi-model Tier 1 score	Measured	Complete-table top-candidate accuracy recorded as 88.66%.
20-feature neural selector	Measured	Train 94.78%; validation 90.80% on 87 predictions.
60-second live prediction	Attempted, failed	Zero live candidate rows.
Nearest-historical-node fallback	Attempted, failed	Fallback run still reported no candidates.
Adaptive edge and selector feedback	Implemented code	Script and output schema exist; no preserved completed-run metrics claimed.
Full multi-hop path enumeration	Specified extension	Not demonstrated by the recorded next-node experiment.
Expected shortest–longest cost bound	Framework formalization	Valid for a finite feasible path set and scalar path cost.
Time-decayed pattern learner	Specified extension	Frequency/adaptive tables exist; no separate calibrated sequence learner demonstrated.
Contour-derived terrain cost	Specified extension	Equations and workflow defined; no aircraft or field implementation demonstrated.
Random-start/goal simulator and Team A/B UI	Specified extension	Interaction requirements were discussed; no measured UI experiment is claimed.
Finance and allies/safe locations	Specified extension	Contextual design variables; absent from aircraft feature list.
Military, SAR, IT, and logistics transfer	Proposed application	Requires independent domain data, constraints, training, and validation.
98% accuracy	Unsupported	No recorded experiment achieved or tested this threshold.

C Reproducibility checklist

Before a revised empirical paper is submitted, archive the following:

- exact source files and commit hash for every pipeline stage;
- schemas and row counts for every input and output table;
- immutable training, validation, and test identifiers;
- collection windows, API responses, rate limits, and exclusion rules;
- trained weight files and normalization parameters;
- baseline predictions on the same held-out split as the neural selector;
- node, edge, path, geographic, ranking, and calibration metrics;
- error examples, missing-data tests, drift tests, and live shadow results; and
- a precise AI-use declaration and independent code review.